



Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia San Juan Bosco

TRABAJO FINAL
PROGRAMACIÓN ORIENTADA A OBJETOS

AÑO 2026



1. Introducción y Objetivos

El presente trabajo final constituye la Instancia Supervisada de Formación Práctica Profesional (ISFPP) de la asignatura. Su propósito es la integración y aplicación de los conocimientos teóricos y prácticos adquiridos durante la cursada para el desarrollo de una aplicación de software de mediana complejidad.

Los objetivos principales de esta instancia son:

- Dominar el Paradigma Orientado a Objetos, aplicando correctamente los principios de abstracción, encapsulamiento, herencia y polimorfismo.
- Aplicar patrones de diseño como soluciones reutilizables a problemas comunes en el diseño de software.
- Desarrollar una aplicación funcional que resuelva un problema concreto, contemplando desde la lógica de negocio hasta la interacción con el usuario y la persistencia de datos.
- Incorporar competencias en el uso de herramientas modernas de desarrollo, como sistemas de control de versiones (Git/GitHub) y entornos de desarrollo integrados (VSCode/Eclipse).
- Fomentar prácticas profesionales a través del trabajo en equipo, la gestión de proyectos y la defensa técnica de las soluciones implementadas.

2. Requerimientos Mínimos del Sistema

El sistema a desarrollar debe ser un juego que cumpla con los siguientes requerimientos técnicos y conceptuales.

Aplicación del Paradigma Orientado a Objetos: El diseño del juego debe estar centrado en objetos, demostrando un uso claro y justificado de:

- Abstracción y Encapsulamiento: Clases bien definidas con responsabilidades únicas y protección adecuada de sus datos.
- Herencia: Uso de jerarquías de clases para reutilizar código y modelar relaciones "es un".
- Polimorfismo: Implementación de comportamiento polimórfico a través de herencia o interfaces.

Patrones de Diseño: Es indispensable la correcta implementación de los siguientes patrones:

- Modelo-Vista-Controlador (MVC): Para separar la lógica de negocio, la representación de los datos y la interfaz de usuario.
- Data Access Object (DAO): Para abstraer y encapsular el acceso a la base de datos.



- Singleton: Para garantizar una única instancia de clases que lo requieran (ej. gestores de conexión con base de datos).
- Patrón Adicional: Adicionalmente, el equipo deberá seleccionar e implementar al menos un patrón de diseño más (ej. Factory, Observer, Decorator, Strategy, etc.), justificando su elección en la documentación y defensa del proyecto.

Interfaz Gráfica de Usuario (IGU): La aplicación debe contar con una IGU desarrollada en Java. La interfaz debe ser intuitiva y centrarse en una experiencia de usuario fluida y coherente.

Persistencia de Datos: El sistema deberá gestionar datos de forma persistente a través de una base de datos relacional.

Concurrencia (Hilos): Se deberá implementar el uso de hilos (threads) para manejar tareas que podrían afectar la responsividad de la IGU. Por ejemplo, consultas a la base de datos que puedan demorar, procesamiento de archivos en segundo plano o tareas programadas.

3. Metodología de Trabajo y Entregables

Los proyectos se realizarán en equipos de cuatro (4) integrantes, a definir por los mismos alumnos.

Antes de iniciar la codificación, cada equipo deberá entregar un Plan de Proyecto (ver Anexo A para la plantilla), excluyendo el punto 3 y el diagrama UML, estos serán completados una vez visto estos temas. Este documento es de entrega obligatoria y debe ser aprobado por la cátedra para poder continuar.

El proyecto se gestionará en un repositorio privado de GitHub. Todos los integrantes deben contribuir activamente. Se evaluará el historial de commits, esperando que estos sean atómicos, frecuentes y con **mensajes descriptivos**. La autoría de los commits deberá ser claramente identificable para cada miembro del equipo.

A lo largo del periodo lectivo, se establecerán diversas instancias de control con el fin de supervisar el progreso (ver planificación de la materia). En cada una de estas etapas se exigirá la presentación de las novedades del desarrollo del software, abarcando desde la elaboración inicial del plan de trabajo y la configuración del repositorio Git respectivo, hasta la conclusión definitiva de la propuesta.

4. Defensa del Proyecto

La aprobación requiere la defensa oral del trabajo realizado ante la cátedra. Cada equipo dispondrá de un turno de 45 minutos en total, en la cual se evaluará el proyecto y cada uno de los integrantes. Los turnos de presentación para cada grupo se asignarán de forma aleatoria durante las fechas asignadas a las evaluaciones (ver planificación de la materia).



En la presentación se espera que todos los miembros del equipo participen activamente, deberá cubrir:

- **Introducción:** Presentación del equipo y del problema que resuelve el sistema.
- **Arquitectura y Diseño:** Explicación de la arquitectura general (MVC), diagramas de clases (UML) que muestren las relaciones clave y justificación de los patrones de diseño implementados (especialmente el patrón adicional elegido).
- **Demostración Funcional:** Recorrido en vivo por las principales funcionalidades de la aplicación.
- **Conclusiones:** Breve resumen de los logros, los desafíos encontrados y posibles mejoras a futuro.
- **Preguntas:** sobre los conceptos teóricos, aplicados en su trabajo final.

5. Criterios de Evaluación

La evaluación será integral, considerando tanto las entregas indicadas en la planificación como el producto final del proceso de desarrollo, cada entrega durante la cursada representa un 10% de la nota, el 50% restante corresponde a la defensa final. **Un sistema que funcione no garantiza la aprobación si no aplica correctamente los conceptos de la materia.**

- Correctitud y aplicación de conceptos POO: Calidad del diseño de clases, uso de herencia y polimorfismo, encapsulamiento.
- Calidad del código y arquitectura: Implementación de patrones de diseño, legibilidad del código, estructura del proyecto, manejo de excepciones.
- Funcionalidad y completitud del sistema: Cumplimiento de los requerimientos y robustez de la aplicación.
- Gestión del proyecto y trabajo en equipo: Calidad del plan de proyecto, uso de Git/GitHub y colaboración demostrada.
- Calidad de la presentación y defensa: Claridad en la exposición, dominio del tema y calidad de las respuestas.

6. Consideraciones Adicionales

Abandono de Integrantes: Si un integrante abandona el grupo en una etapa temprana, el equipo podrá unirse a otro grupo en situación similar (sin superar el límite de 4 integrantes) o continuar con una reducción del alcance, previa aprobación de la cátedra. En etapas avanzadas, el equipo deberá solicitar una redefinición del alcance y funcionalidades para ajustarse a la nueva conformación.

Integridad Académica: Se esperan trabajos originales. Cualquier indicio de plagio, tanto de fuentes externas como de otros grupos, resultará en la desaprobación automática de la asignatura para todos los integrantes involucrados. Se puede utilizar asistencia de IA pero cada uno debe poder explicar lo desarrollado.



Anexo A: Plantilla del Plan de Proyecto

A continuación se presenta un ejemplo en formato Markdown, el necesario para el README.md de su repositorio.

Proyecto: Tower Defense - El Último Bastión

1. Integrantes del Equipo

- Apellido, Nombre
- Apellido, Nombre
- Apellido, Nombre
- Apellido, Nombre

2. Dominio y Alcance del Sistema

Descripción del Problema

Se busca desarrollar una aplicación de escritorio del clásico videojuego **Tower Defense**. El jugador debe utilizar recursos limitados para construir torres defensivas en un mapa con un camino predefinido. Oleadas de enemigos avanzarán por este camino, y el objetivo es destruir a todos los enemigos antes de que lleguen al final del recorrido.

Objetivo del Sistema

El sistema será un juego funcional y extensible que permitirá al jugador experimentar las mecánicas básicas del género. El diseño debe ser modular para facilitar la adición de nuevos tipos de torres, enemigos o mapas en el futuro, aplicando rigurosamente los conceptos del paradigma orientado a objetos.

Funcionalidades Principales (Features)

- **Gestión de Torres:**
 - El jugador puede seleccionar distintos tipos de torres desde un panel.
 - El jugador puede posicionar las torres en ubicaciones válidas del mapa.
 - Cada torre tiene un costo, un rango de ataque y un daño específico.
- **Sistema de Oleadas de Enemigos:**
 - Los enemigos aparecen en oleadas de dificultad creciente.
 - Existen diferentes tipos de enemigos (ej: rápidos pero débiles, lentos pero resistentes).
 - Los enemigos siguen un camino preestablecido en el mapa.
- **Mecánicas de Juego:**
 - El jugador cuenta con una cantidad inicial de "recursos" (oro) para construir torres.
 - Destruir enemigos otorga recursos adicionales.



- El jugador tiene un número limitado de "vidas", que disminuyen si un enemigo llega al final del camino. El juego termina si las vidas llegan a cero.
- ****Interfaz Gráfica (IGU):****
 - Visualización del mapa, el camino, las torres y los enemigos.
 - Panel de control para seleccionar torres, ver recursos, vidas y número de oleada.
 - Botón para iniciar la siguiente oleada.
- ****Persistencia:****
 - Sistema de guardado y carga de los puntajes más altos (High Scores) en una base de datos.

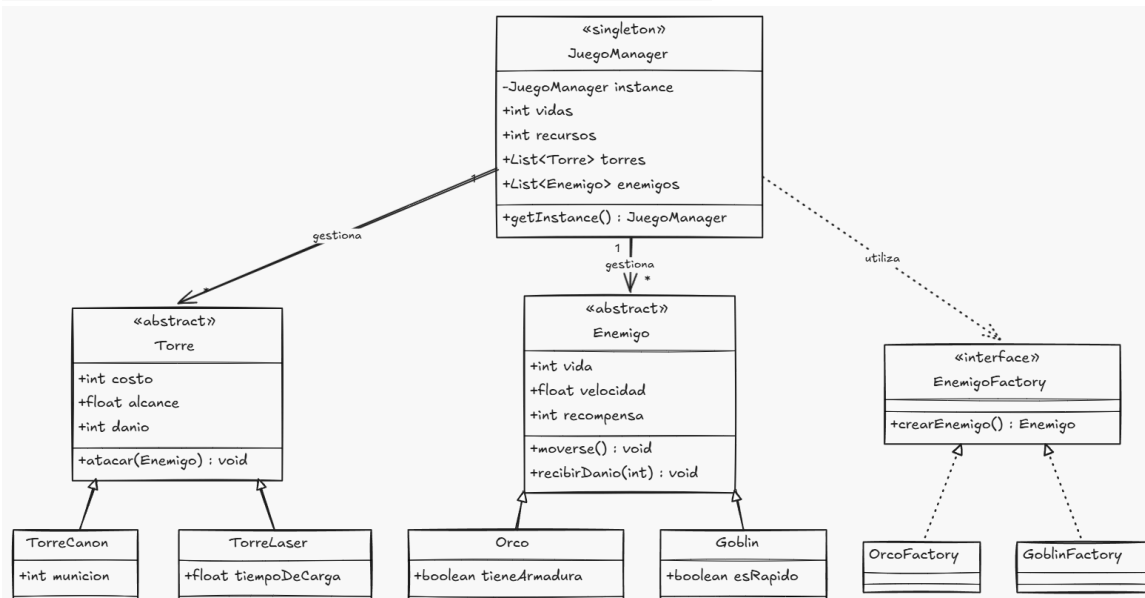
3. Arquitectura y Diseño

Patrón de Diseño Adicional: Factory Method

- ****Nombre del Patrón:**** ****Factory Method (Método de Fábrica)****.
- ****Justificación:**** Se utilizará este patrón para la creación de los objetos `Enemigo`. Tendremos una clase abstracta `EnemigoFactory` con un método `crearEnemigo()`. Se crearán subclases concretas como `OrcoFactory` y `GoblinFactory` que implementarán este método para instanciar los enemigos correspondientes. Esto ****desacopla**** la lógica del juego (que solo necesita pedir un enemigo de un tipo) de la lógica de instanciación de cada enemigo concreto. Así, añadir un nuevo tipo de enemigo (ej. `DragonFactory`) no requerirá modificar el código que gestiona las oleadas.

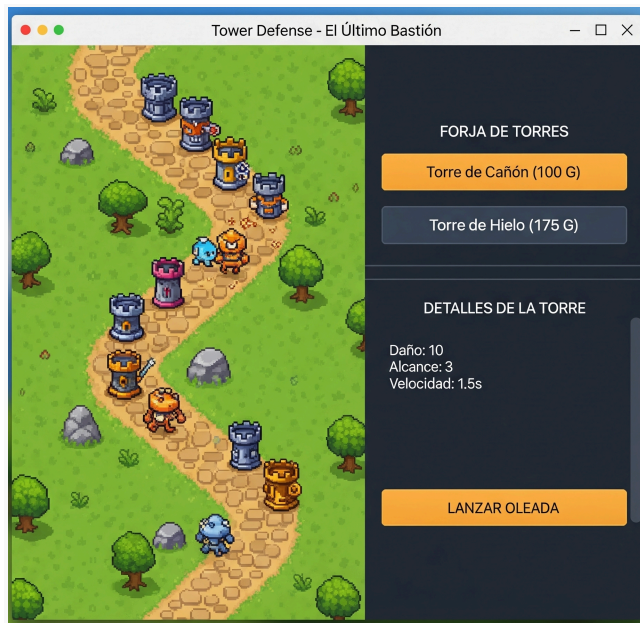
Diagramas de Diseño

****Diagrama de Clases UML (Conceptual)****





Prototipo de la IGU (Wireframe)



4. Stack Tecnológico

- **Lenguaje:** Java 17
- **IDE:** Visual Studio Code
- **Base de Datos:** MySQL 8.0 (para persistencia de High Scores)
- **Framework de IGU:** Java Swing
- **Control de Versiones:** Git y GitHub Classroom